

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Nástroj pro automatickou kontrolu programátorských
úkolů



2026

Vedoucí práce:
Mgr. Petr Osička, Ph.D.

My Linh Duongová

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor: My Linh Duongová
Název práce: Nástroj pro automatickou kontrolu programátorských úkolů
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informatika, Specializace: Programování a vývoj software
Vedoucí práce: Mgr. Petr Osička, Ph.D.
Počet stran: 20
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: My Linh Duongová
Title: Automated checker for programming assignments
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Computer Science, Specialization: Programming and Software Development
Supervisor: Mgr. Petr Osička, Ph.D.
Page count: 20
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Při náboru na stážistické pozice ve firmách se programátorské úkoly uchazečů často kontrolují manuálně, což je při vysokém počtu zájemců časově velmi náročné. Práce na tento problém reaguje vývojem a implementací platformy InQuest pro firmu Edhouse, která celý proces validace automatizuje. Systém si po zadání odkazu na GitHub repozitář kód bezpečně stáhne a spustí ho v izolovaném kontejneru přes Podman nebo Docker. Zde ověří strukturu složek a zachytí výstupy z kompilace i samotného běhu programu. Řídící aplikace je napsaná v C# (.NET) a pro spolehlivou komunikaci s testovacími kontejnery využívá frontu zpráv RabbitMQ, přičemž výsledné reporty jsou přes API dostupné pro potřeby firmy.

Synopsis

When recruiting for internship positions in companies, the programming tasks of applicants are often checked manually, which is very time-consuming when there is a large number of applicants. This work responds to this problem by developing and implementing the InQuest platform for the company Edhouse, which automates the entire validation process. After entering a link to the GitHub repository, the system safely downloads the code and runs it in an isolated container via Podman or Docker. Here, it verifies the folder structure and captures the outputs from compilation and the program run itself. The control application is written in C# (.NET) and uses the RabbitMQ message queue for reliable communication with test containers, with the resulting reports available for the company's needs via API.

Klíčová slova: automatizované testování, Podman, kontejnerizace, RabbitMQ, GitHub API

Keywords: automated testing, Podman, containerization, RabbitMQ, GitHub API

Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	7
1.1	Motivace	7
1.2	Cíl práce	7
2	Analýza existujících řešení	7
3	Dostupné nástroje a technologie	7
3.1	C# .NET	7
3.1.1	Knihovny	8
3.1.1.1	Entity Framework Core	8
3.1.1.2	AutoMapper	8
3.2	Podman	8
3.3	RabbitMQ	8
4	Architektura a návrh systému	8
4.1	Clean Architecture	8
4.2	Backend	9
4.2.1	Prezentační vrstva	9
4.2.2	Dto	9
4.2.3	Doména	9
4.2.4	Aplikace	10
4.2.5	Infrastruktura	10
4.3	Runners	10
4.4	.gitlab-ci.yml	11
5	Uživatelská dokumentace	11
6	Sazba částí dokumentu	11
6.1	Sazba úvodní strany či obsahu	11
6.2	Závěry	12
6.3	Sazba literatury	12
6.4	Drobná makra	12
6.5	Sazba rejstříku	12
6.6	Sazba zdrojových kódů	14
	Závěr	16
	Conclusions	17
A	První příloha	18
B	Druhá příloha	18
C	Obsah elektronických dat	18

Seznam tabulek

1	Odstavce v tabulkách	13
---	--------------------------------	----

1 Úvod

1.1 Motivace

Firma Edhouse, zabývající se vývojem software a hardware, přijímá každým rokem nové stážisty. Kvůli velkému zájmu ale vyhodnocování vstupních programátorských úkolů začalo být neudržitelné. Nejen že museli mentoři úkoly manuálně stahovat a testovat, museli kontrolovat úkoly v různých jazycích. Tento proces zdržoval náborové řízení a bral hodně času programátorům od jejich práci. Motivací aplikace inQuest je tedy proces kontroly stážistických úkolů zautomatizovat, aby se mentoři mohli zabývat až posudkem kvality kódu.

1.2 Cíl práce

Cílem bakalářské práce je navrhnout a vyvinout systém pro kontrolu programátorských úkolů. Výsledný systém musí být schopen autonomně stáhnout kód z GitHub repozitáře, detekovat programovací jazyk a v izolovaném kontejnerovém prostředí bezpečně ověřit strukturu projektu, úspěšnost kompilace a funkčnost běhu programu. Technickým cílem je postavit robustní backend v C# (.NET) s asynchronní architekturou využívající RabbitMQ.

Hlavní cíle tedy jsou:

- Analýza požadavků: Definovat potřeby firmy ohledně struktury testovaných úkolů a formátu výsledných reportů.
- Izolace a validace: Navrhnout kontejnerové prostředí, které zajistí bezpečné sestavení, spuštění a analýzu cizího kódu bez rizika pro firemní infrastrukturu.
- Asynchronní architektura: Propojit řídicí aplikaci v C# (.NET) s exekucí kontejnery pomocí message brokera RabbitMQ.
- Nasazení a integrace: Připravit API rozhraní pro snadné poskytování výsledných reportů.

2 Analýza existujících řešení

3 Dostupné nástroje a technologie

3.1 C# | .NET

Backend je implementován v programovacím jazyce C# na platformě .NET8. .NET je bezplatná multiplatformní opensourcová vývojářská platforma pro vytváření mnoha druhů aplikací od společnosti Microsoft.

3.1.1 Knihovny

3.1.1.1 Entity Framework Core

Entity Framework Core (EF Core) je oficiální, open-source a cross-platform Object-Relational Mapper (ORM). Slouží jako moderní nástroj pro přístup k datům, který mapuje C# objekty na relační databáze a eliminuje potřebu psát většinu ručního kódu pro komunikaci s databází.

3.1.1.2 AutoMapper

AutoMapper je open-source knihovna pro C# a .NET, která slouží jako objektový mapper k automatizaci převodu dat mezi různými typy objektů. Eliminuje nutnost psát redundantní kód pro ruční kopírování vlastností, například při mapování entit z databáze na DTO (Data Transfer Objects) nebo ViewModels.

3.2 Podman

Vzhledem k tomu, že systém zpracovává a spouští neověřené zdrojové kódy od externích uchazečů, existovalo riziko spuštění malwaru nebo nekonečných smyček. Z tohoto důvodu byla představena kontejnerizace. Podman (s alternativní možností Dockeru) je kontejnerizační nástroj pro správu kontejnerů. Narozdíl od Dockeru funguje v tzv. rootless režimu, což znamená, že kontejnery nepotřebují oprávnění správce za svého běhu. Dále nevyužívá centrální fázový proces (daemon) a díky tomu snižuje spotřebu zdrojů. Podman je kompatibilní se standardy OCI (Open Container Initiative) a jeho CLI rozhraní je podobné s Dockerem, jehož příkaz `docker` je nahrazen `podman`.

3.3 RabbitMQ

RabbitMQ je open-source message broker který umožňuje aplikacím a službám asynchronně a spolehlivě vyměňovat si data mezi sebou. Je postaven na protokolu AMQP (Advanced Message Queuing Protocol), ale podporuje také další protokoly jako MQTT, STOMP nebo HTTP, což zajišťuje širokou kompatibilitu s různými technologiemi a jazyky. Po úspěšném doběhnutí předá testovací kontejner výsledná data do fronty RabbitMQ. Vyvíjená aplikace tyto zprávy následně odebírá a zpracovává.

4 Architektura a návrh systému

4.1 Clean Architecture

Česky „Čistá architektura“ je návrhový vzor, který definuje strukturu softwarového systému tak, aby byla nezávislá na frameworku, UI, databázi a externích

agentech. Díky oddělení byznysové logiky a technických detailů, zajišťuje snadnou údržbu, testovatelnost a abstrakci mezi vrstvami. Podle striktního pravidla závislostí (angl. The Dependency Rule) je směr vrstev definován do středu. Vnitřní vrstvy nemají povědomí o tom, co se děje ve vnějších vrstvách. Vrstvy se dělí na následovné:

- **Doménová** (Domain/Entity): nachází se v samotném jádře architektury. Při externích změnách není zde očekávaná změna. Nemá žádné externí závislosti a obsahuje jen objekty, rozhraní a funkce.
- **Aplikační** (Application/Use Cases): definuje případy užití aplikace. Zahrnuje služby, dotazy a validátory.
- **Infrastrukturní** (Infrastructure): vrstva, která obsahuje databáze nebo externí komponenty.
- **Prezentační** (Presentation/API): vystavuje koncové body pro komunikaci přes HTTP. Jejím úkolem je přijímat požadavky, předávat je aplikační vrstvě ke zpracování a zobrazovat výsledky.

4.2 Backend

Podle principů Čisté architektury je backend rozčleněný na následující vrstvy/složky: *prezentační*, *.Application*, *.Domain*, *.DTO*, *.Infrastructure*

4.2.1 Prezentační vrstva

Ve vrstvě (*Edhouse.inQuest*) se inicializuje Swagger, jako interaktivní RESTová dokumentace, a registrují se všechny služby pro Dependency Injection. Pro logování chování aplikace se využívá framework Serilog.

Projekt obsahuje složky *Controllers*, jejíž úkolem je řízení kontrolerů a jejich koncových bodů, *Filters*, která zachycuje výjimky a nakonec *Mapper*, kde se převádí objekty aplikace na DTO, nebo databázové entity naopak.

4.2.2 Dto

Vrstva datových přenosových objektů představuje unifikovaný datový kontrakt pro celý systém. DTO jsou reprezentovány jako třídy bez byznys logiky, které přenášejí data mezi jednotlivými vrstvami a externími systémy.

4.2.3 Doména

Doménová vrstva v projektu obsahuje byznys model nezávislý na jakýchkoli frameworkcích, databázích či externích knihovnách. Zatímco ostatní vrstvy, kromě DTO vrstvy, mohou záviset na této, doménová nezávisí na jiné. V tomto specifickém systému existují projekty modelů pro abstrakce (rozhraní služeb), výjimky, nastavení a tak.

4.2.4 Aplikace

Aplikační vrstva obsahuje orchestraci datových toků a validační logiku procesů. Obsahuje tři složky s případy užití.

- **/AssignmentEvaluation** slouží k parsování poskytnutého URL ke Github repozitáři, zjistí se programovací jazyk pomocí GitHub API
- **/Services** je popis poskytovaných služeb.
 - *Assignment*: spravuje životní cyklus assignmentu a stav ukládá do databáze.
 - *Result*: zpracovává výstup po vyhodnocení, zparsuje stdout na build/run výstup, nastavuje příznaky úspěchu a ukládá výsledek do databáze.
 - *Value*: kontroluje hodnoty výstupu. Pro každé zadání by se měl tento modul měnit.
- **/Validator** je složka pro služby, které se týkají validace.

4.2.5 Infrastruktura

Infrastrukturní vrstva zajišťuje komunikaci aplikace s externími komponentami a databází. Mezi její hlavní součásti patří *Entity Framework Core* pro správu databáze. Dále se zde nachází *PodmanManager*, který komunikuje se službou Podman přes HTTP REST rozhraní. Ten řídí spouštění a mazání kontejnerů. Infrastruktura obsahuje napojení na message broker *RabbitMQ*, jenž zajišťuje asynchronní příjem zpráv z kontejnerů a jejich předání na zpracování. A nakonec *GitHub API* pro zjištění jestli GitHub repozitář existuje a pro získání jeho programátorského jazyka.

4.3 Runners

Klíčovou součástí systému inQuest je sada exekučních běžců (runners) umístěných ve složce *runners*. Struktura je rozdělena na základní shell skripty a dedikované složky pro jednotlivé technologické platformy. Složka *baseScripts* zahrnuje skripty, které definují běh pro testované jazyky. Skládá se z následujících skriptů:

- **installPrerequisites.sh**: tento skript se nevyužívá za běhu kontejneru, ale zajišťuje instalaci dodatečných systémových nástrojů při vytváření obrazu. Stahuje git, jq pro manipulaci s JSON datami, curl pro odeslání dat a dos2unix pro konverzi řádkových konců mezi formáty specifické pro UNIX a Windows.
- **checkVariables.sh**: skript, který kontroluje přítomnost potřebných proměnných prostředí (environment variables) při startu kontejneru. Prvně

jestli byl přiložen URL ke GitHub repozitáři a poté jestli má kontejner přístup k cestě pro access token. Ten je nezbytný jestli je repozitář soukromý. Ale i kdyby byl veřejný, je potřeba nastavit access token s podman secrets.

- **generateValidationResult.sh** a **reportResult.sh**: tyto skripty se starají o vygenerování a zformátování DTO s informací ohledně průběhu projektu uchazeče a jeho následného odeslání pomocí služby RabbitMQ do zprávové fronty.
- **validate.sh**: obsahuje logiku pro stahování GitHub repozitáře, vyjmutí informací, spuštění konkrétní implementace jazykového skriptu *start.sh* s timeoutem a vypsání následných zpráv po ukončení běhu projektu.
- **run.sh**: hlavní řídicí skript, který orchestruje a spouští ostatní skripty.

Konkrétní implementace pro jednotlivé jazykové se nachází pod složkami s jejich zkratkou. Využívá se výše popsany základ se svou vlastní logikou a balí je do výsledného kontejnerového obrazu. Každá složka obsahuje:

- **runnerScript-<language_extension>.sh**: tento skript reprezentuje konkrétní implementaci daného jazyka. Kontroluje přítomnost povinných souborů v projektu a volá specifické příkazy technologie ke kompilaci a spuštění běhu.
- **Dockerfile**: definuje předpis pro sestavení obrazu. Přibalí se zde sdílené skripty, stáhnou se dodatečné systémové příkazy dle *../installPrerequisites.sh*, přejmenuje se skript daného jazyka na *start.sh* a nastaví se ENTRYPOINT na skript *run.sh*

Díky této architektuře je proces přidání nového jazyka velice snadné. Detailní popis je v dokumentaci nacházející se v *documentation/testRunnerDocumentation.md*.
TODO/Poznámka: mám rozebrat/zkopírovat dokumentaci sem?

4.4 .gitlab-ci.yml

5 Uživatelská dokumentace

6 Sazba částí dokumentu

6.1 Sazba úvodní strany či obsahu

Vysázení všech podstatných částí úvodu práce obstará makro `\maketitle`. Pro správné vysázení všech částí a meta-informací je potřeba použít makra `\title`, `\author` a další. Jejich přehled lze najít ve zdrojovém souboru tohoto dokumentu. V případě použití **pdf** výstupu se generuje i dodatečná hlavička souboru s meta-informacemi jako je autor dokumentu, název práce či dalšími.

6.2 Závěry

Závěr práce by se měl poskytnout jak v původním jazyce práce, tak v jazyce anglickém. Pro sazbu závěru jsou k dispozici příslušná makra. Berte na vědomí, že , se aktivuje plně anglická sazba se všemi konvencemi. Tedy je třeba používat anglické uvozovky a další správné typografické prvky.

```
1 % Tiskne český závěr práce.
2 \begin{kiconclusions}
3 Závěr práce v \uv{českém} jazyce.
4 \end{kiconclusions}
5
6 % Tiskne anglický závěr práce.
7 \begin{kiconclusions}[english]
8 Thesis conclusions written in \uv{English}.
9 \end{kiconclusions}
```

Zdrojový kód 1: Sazba závěrů

6.3 Sazba literatury

Pro sazbu literatury má uživatel dvě možnosti. Může použít služeb balíků `BIBLATEX`, který je pro `kistyles` zapnutý, či lze použít manuální sazbu bibliografie.

6.4 Drobná makra

Základní styl definuje hned několik maker pro usnadnění práce. Například makro `\buno` vysází řetězec „bez újmy na obecnosti“. Je k dispozici i verze s prvním velkým písmenem, `\Buno`.

Je rovněž možno přidávat položky do seznamu zkratk. K tomu slouží makro `\newacronym`, které lze použít například jednoduše jako `\newacronym{UPOL}{UPOL}{\kitextunivcz}`. Na danou zkratku se pak lze odkazovat jednoduše, `\gls{UPOL}`.

Sazba uvozovek respektuje nastavení částí dokumentu, a proto se doporučuje používat makro `\uv`. V anglické závěru práce toto platí taky, viz tato PDF ukázka.

Styl podporuje sazbu odstavců v tabulkách, více obsahuje tabulka [1](#).

K dispozici jsou také makra pro sazbu C# (`\csharp`) či C++ (`\cpp`).

6.5 Sazba rejstříku

Sazba rejstříku sestává z několika kroků:

1. Je třeba přes volbu `index=true` rejstříkování povolit.
2. Použitím makra `\index` rejstříkovat vybrané pojmy.

Tabulka 1: Odstavce v tabulkách

Donec et nisl id sapien blandit mattis. Aenean dictum odio sit amet risus. Morbi purus. Nulla a est sit amet purus venenatis iaculis. Vivamus viverra purus vel magna. Donec in justo sed odio malesuada dapibus. Nunc ultrices aliquam nunc. Vivamus facilisis pellentesque velit. Nulla nunc velit, vulputate dapibus, vulputate id, mattis ac, justo. Nam mattis elit dapibus purus. Quisque enim risus, congue non, elementum ut, mattis quis, sem. Quisque elit.	Etiam suscipit aliquam arcu. Aliquam sit amet est ac purus bibendum congue. Sed in eros. Morbi non orci. Pellentesque mattis lacinia elit. Fusce molestie velit in ligula. Nullam et orci vitae nibh vulputate auctor. Aliquam eget purus. Nulla auctor wisi sed ipsum. Morbi porttitor tellus ac enim. Fusce ornare. Proin ipsum enim, tincidunt in, ornare venenatis, molestie a, augue. Donec vel pede in lacus sagittis porta. Sed hendrerit ipsum quis nisl. Suspendisse quis massa ac nibh pretium cursus. Sed sodales. Nam eu neque quis pede dignissim ornare. Maecenas eu purus ac urna tincidunt congue.	Etiam pede massa, dapibus vitae, rhoncus in, placerat posuere, odio. Vestibulum luctus commodo lacus. Morbi lacus dui, tempor sed, euismod eget, condimentum at, tortor. Phasellus aliquet odio ac lacus tempor faucibus. Praesent sed sem. Praesent iaculis. Cras rhoncus tellus sed justo ullamcorper sagittis. Donec quis orci. Sed ut tortor quis tellus euismod tincidunt. Suspendisse congue nisl eu elit. Aliquam tortor diam, tempus id, tristique eget, sodales vel, nulla. Praesent tellus mi, condimentum sed, viverra at, consectetur quis, lectus. In auctor vehicula orci. Sed pede sapien, euismod in, suscipit in, pharetra placerat, metus. Vivamus commodo dui non odio. Donec et felis.
---	--	--

3. Kompilovat s použitím utility `makeindex`. Pro specifiká tohoto kroku si stačí prohlédnout soubor `Makefile`.

Makro `\index` je redefinováno tak, že sází klikací odkaz na výraz v rejstříku. Je doporučeno jej použít ihned za výrazem^{[1](#)}.

Omezení redefinovaného makra `\index`: klikací odkaz nefunguje, pokud použijete konstrukci `\index{výraz|makro}` (resp. `\index{výraz|(makro)}`), např. `\index{výraz|textit}`.

Rejstřík lze vysázet pomocí makra `\printindex`.

6.6 Sazba zdrojových kódů

Styl nabízí dva způsoby sazby zdrojových kódů:

1. Sazbu řádkových kódů, například **`background-color: white;`**. K tomu slouží makro formátu `\kiinlinecode{jazyk}{separátor}{kód}`. Za separátor je vhodné volit jakýkoliv znak, který se nevyskytuje v samotném sázeném zdrojovém kódu. Za jazyk je nutno dosadit jeden z těchto: C, TeX, PHP, HTML, Lisp, SQL, TeX, Python, Java, TutorialD, text, csharp, cpp, JavaScript, CSS.
2. Sazbu zdrojových kódů do separátních prostředí. Takto vytištěný kód se objeví v seznamu zdrojových kódů. Ukázka například zdrojový kód [2](#). Ukázku sazby naleznete ve zdrojovém kódu tohoto dokumentu.

```
1 int main("cs acsa") // komentar
2 int main("cs acsa") // komentar
3 int main("cs acsa") // komentar
4 int main("cs acsa") // komentar
5 int main("cs acsa") // komentar
```

Zdrojový kód 2: C++

```
1 new object() // komentar
```

Zdrojový kód 3: JS

```
1 public static int main("cs acsa") // komentar
```

Zdrojový kód 4: C#

```
1 SELECT * FROM table_1; /* komentar */
```

Zdrojový kód 5: SQL

```
1 table_1 AND table_2;
```

Zdrojový kód 6: TutorialD

Závěr

Závěr práce v „českém“ jazyce.

Conclusions

Thesis conclusions in “English”.

A První příloha

Text první přílohy

B Druhá příloha

Text druhé přílohy

C Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. .txt) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,

- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwareového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

